

HACKATHON PROJECT GUIDE

Project 1: The CVE-to-My-Stack Translator

Duration	5.30 hours maximum
Skill level	Intermediate (some Python experience required)
Team size	3 to 5 participants recommended
Context	Cyber security education and SMB vulnerability management
Connectivity	Data sources link provided in problem document

1. Project Overview

Every working day, dozens of new CVEs (Common Vulnerabilities and Exposures) are published. A small IT team cannot read them all, let alone work out which ones apply to the specific software versions and configurations they actually run. The result is a firehose of security information that produces either paralysis or blanket patching, both of which waste time.

This project challenges you to build a tool that solves a real and pressing problem: given a list of software assets, tell the user which published vulnerabilities are relevant to them, how urgent each one is, and what to do about it. The output should convert a flood of CVEs into a short, prioritised action list with plain-English explanations.

The core problem in one sentence

A small IT administrator cannot filter hundreds of daily CVEs to the three or four that actually affect their systems. Your tool does this for them.

2. Key Terminology

The following terms are used throughout this guide and will be central to your build. Understanding them clearly is essential before you begin.

Term	Definition
CVE	Common Vulnerability and Exposure. A unique identifier (e.g. CVE-2024-1234) assigned to a publicly known security flaw in a software product.

NVD	National Vulnerability Database. The US government repository maintained by NIST that enriches CVE records with severity scores, affected product lists, and patch references.
CPE	Common Platform Enumeration. A structured naming scheme for software and hardware. CPE identifiers allow CVE records to be matched precisely to specific products and versions (e.g. cpe:2.3:a:microsoft:365_apps:-:*:*:*:*:*:*).
CVSS	Common Vulnerability Scoring System. A numerical score (0 to 10) that describes the severity of a vulnerability. A score of 9 or above is considered Critical.
EPSS	Exploit Prediction Scoring System. A probability score (0 to 1) produced daily by FIRST that estimates how likely a given CVE is to be exploited in the wild within the next 30 days. Unlike CVSS, it reflects real-world threat activity.
KEV	Known Exploited Vulnerabilities. A catalogue maintained by CISA (the US Cybersecurity and Infrastructure Security Agency) listing CVEs confirmed to have been actively exploited. KEV membership is one of the strongest urgency signals available.
Asset list	A list of software products and versions that an organisation runs. In this project, the user provides this as a text file or pasted input. It may use informal names such as 'Office 365' rather than formal CPE identifiers.
Normalisation	The process of mapping informal, inconsistent, or vendor-specific product names to their canonical CPE identifiers. This is the hardest and most important step in the build.
EPSS percentile	The proportion of all CVEs that score at or below a given CVE's EPSS probability. A percentile of 0.95 means this CVE is more likely to be exploited than 95% of all known CVEs.
Fuzzy matching	A technique for finding near-matches between strings. Used here to map user-supplied product names to CPE identifiers even when spelling or formatting differs.

3. Aims and Objectives

3.1 Aim

To build a working tool that takes a list of software assets as input and returns a prioritised, plain-English list of CVEs relevant to those assets, ranked by real-world exploitability.

3.2 Objectives

By the end of the session your team should have:

1. Loaded and parsed at least one of the offline data feeds (NVD CVE data, CISA KEV, or EPSS scores).
2. Built a normalisation function that maps at least 15 to 20 common software names to their CPE identifiers.

3. Implemented a matching function that filters the CVE dataset to entries relevant to the user's asset list.
4. Applied EPSS scores and KEV flags to rank the filtered CVEs by urgency.
5. Produced a structured output (CSV or printed table) that includes CVE ID, affected asset, EPSS score, KEV flag, and a plain-English risk summary.
6. Demonstrated the tool working against a sample asset list provided by the facilitator.

3.3 Stretch Goals

If your core build is complete before time is called, consider one or more of the following:

- Generate a one-page summary brief in addition to the CSV output.
- Add a simple scoring formula that combines CVSS severity and EPSS probability into a single urgency rank.
- Handle version range matching (e.g. matching CVE-2024-1234 which affects versions 3.0 to 3.5 against an asset list entry that gives version 3.2).
- Add a simple command-line interface that accepts an asset list file as an argument.

4. Anticipated Outcomes

A successful build will produce all of the following:

Output	Description
Prioritised CVE table	A ranked list of CVEs relevant to the user's asset list. Columns: CVE ID, affected asset (in the user's own naming), CVSS score, EPSS score, KEV flag (yes/no), and a one-sentence plain-English risk description.
Normalisation dictionary	A Python dictionary or CSV mapping common product names to CPE identifiers, covering at least 15 to 20 widely used SMB software titles.
Working script	A Python script that accepts an asset list and produces the prioritised CVE table. The script should run cleanly against the sample asset list without errors.
Demo output	A short presentation (5 minutes maximum) showing the tool running against the sample asset list and explaining the design decisions your team made.

5. Use Cases and Target Users

Understanding who will use this tool and in what context will help you make design decisions during the build. The following scenarios are derived directly from the project brief.

Use Case 1: The Solo SMB System Administrator

Scenario

Alex is the only IT person at a 40-person accountancy firm. The firm runs Windows Server, Microsoft 365, a third-party payroll application, and a Cisco router.

Alex receives a notification that 47 new CVEs were published overnight. He has no time to read each one.

He pastes his asset list into the tool and receives a table of 4 relevant CVEs, of which one is in the CISA KEV catalogue. He patches that one immediately.

- Primary user: single sysadmin or small IT team at an SMB, school, GP surgery, or charity.
- Value delivered: converts a firehose into a short, actionable list with clear urgency signals.
- Asset list format: likely informal (for example, 'Office 365 Business', 'Windows Server 2019', 'Sage Payroll 22').

Use Case 2: The School or University IT Support Team**Scenario**

A university IT support team manages a mixed estate of Windows and Linux systems, plus a Moodle instance and several research applications.

The team lead wants a weekly patch briefing to share with the CISO. She runs the tool each Monday morning against the estate asset list and exports the output as a CSV, which she attaches to her weekly report.

- Primary user: small to medium higher or further education IT team.
- Value delivered: structured weekly output that is ready to share with management without manual curation.
- Asset list format: likely more structured, possibly derived from an existing CMDB or spreadsheet.

Use Case 3: The Charity or Third-Sector Organisation**Scenario**

A housing charity has no dedicated security team. Their IT is managed part-time by a volunteer with some technical background.

The charity recently completed Cyber Essentials certification and wants to maintain their patch posture. The volunteer runs the tool quarterly to check whether any of the software they certified is now vulnerable.

- Primary user: non-specialist IT volunteer or part-time IT support worker.
- Value delivered: plain-English output that does not require security expertise to interpret.
- Asset list format: informal, possibly just a list of application names and rough version numbers.

6. Data Feeds and How to Use Them

All data for this project is pre-downloaded and provided to you at the start of the session. The four feeds below are your primary assets. You must use these local files rather than calling any live data service.

Important: NVD legacy feeds were retired in August 2025

NIST retired the legacy 1.1 JSON data feed files in August 2025 and replaced them with new 2.0 data feed files. Any tutorial or guide referencing the old `nvd-cve-1.1-YYYY.json.gz` download paths is now out of date.

For this hackathon, the pre-downloaded files use the Fraunhofer FKIE community reconstruction, which mirrors the NVD data in the same per-year JSON format and is the most practical offline source available.

6.1 NVD CVE Data

Official page	nvd.nist.gov/vuln/data-feeds
Recommended source	Fraunhofer FKIE community reconstruction: github.com/fkie-cad/nvd-json-data-feeds
Format	JSON, compressed with xz. One file per year (e.g. <code>CVE-2024.json.xz</code>).
Used for	The primary database of CVE records. Each record contains the CVE ID, description, CVSS score, affected CPE identifiers, and reference links.
Key fields	<code>cve.id</code> , descriptions, metrics (CVSS scores), configurations (CPE applicability statements)
Typical size	Approximately 40 to 60 MB uncompressed per year file.

Official NVD data feeds page: <https://nvd.nist.gov/vuln/data-feeds>

Fraunhofer FKIE reconstruction: <https://github.com/fkie-cad/nvd-json-data-feeds>

Direct download example: <https://github.com/fkie-cad/nvd-json-data-feeds/releases/latest/download/CVE-2024.json.xz>

Decompress with: `xz -d -k CVE-2024.json.xz`

An All feed covering every year is also available from the same releases page. For the hackathon, work with the current year file only unless your sample asset list specifically includes older software.

6.2 CISA Known Exploited Vulnerabilities (KEV) Catalogue

Format	Single JSON file. No registration required.
Used for	Flagging CVEs that have been confirmed as actively exploited in the wild. KEV membership is your strongest urgency signal.
Key fields	cveID, vendorProject, product, shortDescription, dateAdded, knownRansomwareCampaignUse
Typical size	Approximately 1 to 2 MB.
Update frequency	Updated whenever new entries are added, typically weekdays during US Eastern business hours.

Direct JSON download:

https://www.cisa.gov/sites/default/files/feeds/known_exploited_vulnerabilities.json

Catalogue browse page: <https://www.cisa.gov/known-exploited-vulnerabilities-catalog>

Practical tip: load the KEV catalogue into a Python set of CVE IDs at the start of your script. A simple check of whether a CVE ID is in the set is then a single line of code.

6.3 EPSS Scores

Format	Gzip-compressed CSV. Columns: cve, epss (0 to 1), percentile.
Used for	Ranking CVEs by real-world exploitability. A higher EPSS score means a higher probability of exploitation in the next 30 days.
Key fields	cve, epss, percentile
Typical size	Approximately 8 to 12 MB compressed.
Update frequency	Published daily.

Download URL pattern: https://epss.empiricalsecurity.com/epss_scores-YYYY-MM-DD.csv.gz

Example for today: https://epss.empiricalsecurity.com/epss_scores-2026-05-19.csv.gz

Practical tip: load the EPSS data into a dictionary keyed by CVE ID. Lookup is then $O(1)$ when you are processing your matched CVE list.

6.4 CPE Dictionary

Format	XML or JSON (2.0 format).
Used for	Understanding the full list of official product identifiers. Use as a reference when building your normalisation dictionary, not as the primary matching mechanism.

Key fields	cpeName (e.g. cpe:2.3:a:microsoft:365_apps:-:*:*:*:*:*), titles (human-readable product name)
Update frequency	Updated nightly.

Download page: <https://nvd.nist.gov/vuln/data-feeds>

Note for facilitators: the CPE dictionary direct URL is listed on the data feeds page rather than being a static permanent path. Download the file from that page on the morning of the event.

6.5 Pre-Event Download Checklist

The following table is a reference for whoever is preparing the shared drive before the session. All files should be decompressed and ready to use.

Feed	Source URL	Format	Notes
CVE data	github.com/fkie-cad/nvd-json-data-feeds/releases/latest	JSON (.xz)	Decompress before distributing
CISA KEV	cisa.gov/sites/default/files/feeds/known_exploited_vulnerabilities.json	JSON	Single file, ready to use
EPSS scores	epss.empiricalsecurity.com/epss_scores-YYYY-MM-DD.csv.gz	CSV (.gz)	Use today's date in filename
CPE dictionary	nvd.nist.gov/vuln/data-feeds	XML or JSON	Grab link from page on the day

7. Constraints

7.1 Hard Constraints

These are non-negotiable for this session:

- No external APIs. Do not attempt to call the NVD API, FIRST API, CISA API, or any other online data service. All data must be used from the pre-downloaded files provided at the start of the session.
- 5-hour time limit. You must be ready to demo within the session window. Scope your build accordingly.
- Full internet access is available and encouraged. You may use AI tools, search engines, documentation sites, forums, and any other online resources to support your build. The only restriction is that you must not call external vulnerability data APIs (see above). Your local pre-downloaded data files are your data source.

7.2 Scope Constraints

These are practical limitations to keep your build achievable:

- Work with the current year CVE file only, unless specifically told otherwise.
- Your normalisation dictionary needs to cover 15 to 20 product names well, not every product ever made. Breadth is less important than accuracy.
- Do not attempt version range matching in the core build. Exact version matching is sufficient for the MVP. Version ranges are a stretch goal.

7.3 Known Limitations You Should Be Aware Of

These are inherent constraints of the data and approach that you should understand and be able to explain in your demo:

- CVE matching fails silently when product names do not align with CPE identifiers. A CVE that affects your software will simply not appear in results if your normalisation dictionary maps the name incorrectly. This is not an error message; it is a missed result.
- EPSS scores are predictive, not deterministic. A low EPSS score does not mean a CVE is safe; it means exploitation has not been observed at scale yet.
- The CISA KEV catalogue covers confirmed exploitation of specific CVEs. A CVE not in KEV has not necessarily been unexploited; it may simply not have been confirmed or reported.
- NVD enrichment has become selective. As of 2026, NIST prioritises enrichment for CVEs in the KEV catalogue and CVEs affecting federal government software. Older or lower-profile CVEs may have incomplete CPE data.

8. Indicative Technologies

Two technology approaches are supported for this project. Choose the one that best suits your team's skills. Both are equally valid and will be assessed on the same criteria.

8.1 Approach A: Python (Back-end / Data Pipeline)

The Python approach is best suited to teams comfortable with data processing and scripting. It produces a clean, auditable pipeline and is the more straightforward route to the core matching logic.

Library	Install	Purpose
pandas	pip install pandas	Loading, filtering, and sorting the CVE and EPSS CSV/JSON data.
rapidfuzz	pip install rapidfuzz	Fuzzy string matching for normalisation (faster and more accurate than fuzzywuzzy).
json (stdlib)	Built in	Parsing the NVD CVE JSON and CISA KEV JSON files.

gzip / lzma (stdlib)	Built in	Decompressing the .gz EPSS file and .xz NVD files.
csv (stdlib)	Built in	Writing the output CSV.
tabulate	pip install tabulate	Optional. Printing a readable table to the console during development.
Flask	pip install flask	Optional. If your team wants to add a minimal browser-based front end to the Python pipeline.

- Python 3.10 or later.
- A Jupyter notebook is recommended for exploratory work and demonstration, as it allows you to show data at each stage of the pipeline.
- A plain .py script is acceptable for the final submission and may be cleaner for a CLI-style demo.
- No database is required. Keep everything in memory using pandas DataFrames or Python dictionaries.

8.2 Approach B: Web Technologies (Front-end / Full-stack)

The web technologies approach is best suited to teams comfortable with JavaScript and browser-based development. It offers a more visual demo and can handle the data processing entirely in the browser without a server, which is well suited to the offline constraint on the data feeds.

Technology	Role	Notes
HTML / CSS / JavaScript	Core stack	No framework is required. Vanilla JS is entirely sufficient for the data processing and output rendering.
Fuse.js	Fuzzy matching	A lightweight JavaScript fuzzy-search library. Used in place of rapidfuzz for normalisation. Load via CDN: cdn.jsdelivr.net/npm/fuse.js
PapaParse	CSV parsing	Parses the EPSS CSV file in the browser. Load via CDN: cdn.jsdelivr.net/npm/papaparse
React (optional)	UI framework	Optional. Only use React if your team already knows it. A plain HTML page with a file upload input and a results table is entirely acceptable.
Node.js + Express (optional)	Local server	Optional. If your team prefers a back-end approach in JavaScript rather than Python, Node.js with Express can serve the data files locally and run the matching logic server-side.

Practical notes for the web approach:

- The NVD JSON files are large. Load only what you need. Parse the file line by line or filter aggressively in memory rather than rendering the full dataset into the DOM.

- Use the browser File API to allow the user to upload their asset list as a text file, rather than hardcoding it.
- Output can be rendered as an HTML table and also offered as a downloadable CSV using a Blob URL.
- If using CDN-hosted libraries, download them to a local folder before the session begins, in keeping with the no-live-data constraint.

Which approach should my team choose?

If your team is strongest in Python and data processing: choose Approach A.

If your team is strongest in JavaScript and web development: choose Approach B.

If your team is mixed: split the work. Use Python for the data pipeline (loading, matching, ranking) and build a simple HTML front end that reads the Python-generated CSV output.

Both approaches produce the same deliverables and are assessed on the same criteria.

9. Suggested Build Approach

This is a recommended sequence, not a rigid specification. Adapt it to your team's strengths.

Hour 1: Data loading and exploration (0:00 to 1:00)

7. Load the EPSS CSV into a pandas DataFrame. Confirm the columns and check the score range.
8. Load the CISA KEV JSON into a Python set of CVE IDs.
9. Load a single year of NVD CVE JSON (e.g. 2024). Explore the structure. Identify where the CPE identifiers and CVE descriptions live.
10. Load the sample asset list provided by your facilitator.

Hour 2: Normalisation dictionary (1:00 to 2:00)

11. Review the CPE dictionary to understand the naming conventions.
12. Build a Python dictionary mapping at least 15 common product names to their CPE vendor and product strings. For example: 'Office 365' maps to 'microsoft:365_apps', 'Windows 10' maps to 'microsoft:windows_10'.
13. Write a normalisation function that takes a user-supplied product name, applies fuzzy matching against your dictionary, and returns the best CPE match.
14. Test the function against a variety of informal inputs.

Hour 3: CVE matching and filtering (2:00 to 3:00)

15. Write a function that takes a CPE vendor/product string and filters the NVD dataset to CVEs whose configuration blocks reference that CPE.
16. Run this for each asset in the user's normalised asset list.
17. Merge the matched CVE list with the EPSS scores and KEV flags.

Hour 4: Ranking and output (3:00 to 4:00)

18. Sort the merged dataset by EPSS score descending, with KEV entries promoted to the top.
19. Generate a plain-English risk sentence for each CVE. This can be a simple template: 'CVE [ID] affects [product]. EPSS score [X] indicates [low/moderate/high] exploitation probability. [KEV: actively exploited in the wild / Not in KEV.]'
20. Write the output to a CSV file.
21. Print a summary table to the console.

Hour 5: Testing, refinement, and demo preparation (4:00 to 5:00)

22. Run the full pipeline against the sample asset list provided.
23. Fix any errors or edge cases.
24. Prepare your 5-minute demo. Focus on: (a) what the tool does, (b) where the hard problem is (normalisation), and (c) what the output looks like.
25. If time allows, attempt a stretch goal.

10. Sample Asset List

The following sample asset list will be used to test your tool during the demo. Your normalisation dictionary must handle at least 10 of these entries.

Product Name (as supplied)	Version	Notes
Microsoft 365 Apps for Business	Current	Informal name variant
Windows Server 2022	21H2	Server OS
Windows 10 Pro	22H2	Desktop OS
Adobe Acrobat Reader DC	2024.001	PDF reader
Cisco IOS XE	17.9	Router firmware
VMware vSphere	8.0	Virtualisation
Google Chrome	Latest	Browser, version unspecified
OpenSSL	3.0.7	Cryptography library
Apache HTTP Server	2.4.57	Web server
Zoom	5.17	Video conferencing
WordPress	6.4	CMS
Moodle	4.3	LMS

11. Evaluation Criteria

Your demo will be assessed against the following criteria. Each area carries equal weight.

Criterion	What we are looking for
Data pipeline	Can you load and correctly parse at least two of the four data feeds?
Normalisation quality	Does your dictionary produce correct CPE matches for common product names? Can you explain why normalisation is the hardest part of the problem?
Matching accuracy	Does your CVE filter return genuinely relevant results and avoid false positives?
Prioritisation logic	Are EPSS scores and KEV flags applied correctly? Are KEV entries surfaced at the top?
Output clarity	Is the output readable and useful to a non-expert? Could the sysadmin in Use Case 1 act on it without further explanation?
Explanation of limitations	Can your team clearly articulate what the tool gets wrong and why? This demonstrates genuine understanding of the problem.

12. Resources Provided on the Day

The following files will be available on the shared drive at the start of the session. Do not attempt to download replacements during the session.

File	Format	Description
CVE-2024.json	JSON	NVD CVE data for 2024, decompressed and ready to parse.
CVE-2025.json	JSON	NVD CVE data for 2025, decompressed and ready to parse.
known_exploited_vulnerabilities.json	JSON	CISA KEV catalogue, current as of the morning of the event.
epss_scores-[date].csv	CSV	EPSS scores for all CVEs, current as of the morning of the event.
official-cpe-dictionary_v2.3.xml	XML	CPE dictionary for reference. Do not attempt to load this fully; use it to look up specific vendor/product strings.
sample_asset_list.txt	Plain text	The sample asset list from Section 10, ready to load.
starter_notebook.ipynb	Jupyter	A starter notebook with file loading code already written. Use or ignore as you prefer.

A note on the hardest part

The normalisation dictionary is where most teams will spend more time than expected. CVE matching fails silently when names do not align.

A CVE that genuinely affects a product in your asset list will simply not appear in your output if the product name is mapped to the wrong CPE.

There is no error message. There is just a missed result. This is the core challenge of the project, and it is the reason this tool does not already exist as a commodity product.

Spend time on your dictionary. Test it against the sample asset list before you build the rest of the pipeline.